

ModiFylter - 1914546 - Project report

Tommaso Leonardi

Contents

1	Project description and approach	2
2	Implementation and components	2
2.1	The frontend	3
2.1.1	Page loading and authentication	4
2.1.2	Image loading, filter selection and Modify	4
2.1.3	Image download	4
2.2	AWS services	5
2.2.1	AWS Amplify	5
2.2.2	Amazon Cognito	5
2.2.3	Amazon API Gateway	5
2.2.4	Amazon S3	5
2.2.5	AWS Lambda	6
2.2.6	Amazon DynamoDB	6
2.2.7	AWS IAM Identity Center	7
2.2.8	Amazon CloudWatch	7
3	Evaluation	7
3.1	Data	7
3.2	Implementation	8
3.2.1	Puppeteer	8
3.2.2	Docker	9
3.2.3	Amazon ECR	9
3.2.4	Amazon ECS	10
3.3	Results	10
3.3.1	Test 1 (LD jpg)	10
3.3.2	Test 2 (LD png)	11
3.3.3	Test 3 (ND jpg)	12
3.3.4	Test 4 (ND png)	13
3.3.5	Test 5 (HD jpg)	14
3.3.6	Test 6 (LD jpg - high load)	15
4	Conclusion	16

1 Project description and approach

For our project we deployed a serverless image processing application called *ModiFylter*, which provides users the possibility to apply different filters to an image of their choosing (only *grayscale* and *black & white* filters are available for now). Users can request to process a specific image through an intuitive and lightweight web page where they can choose which image to process and which filter to apply, initiate the processing of the image and download the resulting image. Our application is reachable at: <https://main.d36fucso8styeq.amplifyapp.com/>

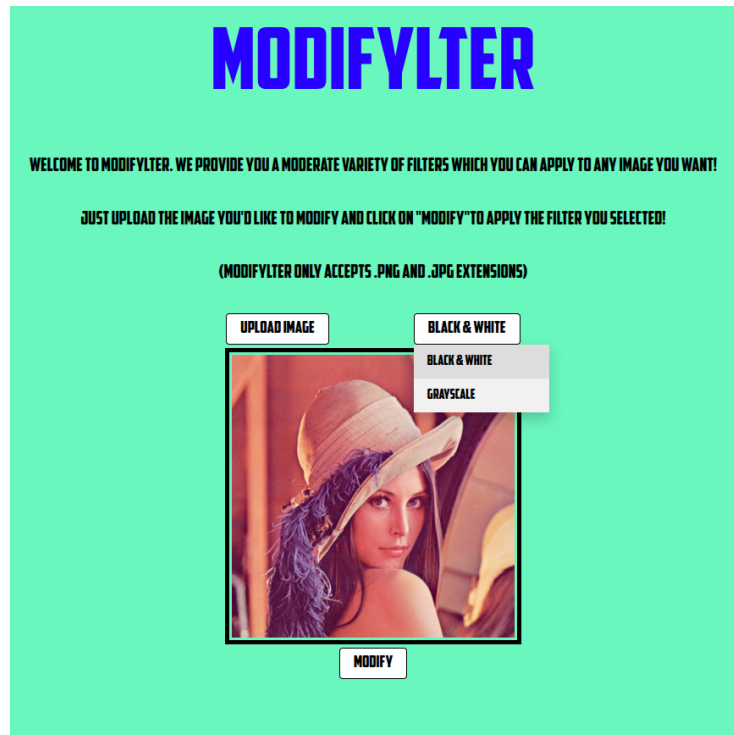


Figure 1: The web page for ModiFylter. The filter to apply can be chosen via a drop-down menu.

We leveraged the technologies available on the *Amazon Web Services* (AWS) platform to deploy our application, which allows for a smoother provisioning of resources (compared to manual provisioning) and effortless server deployment for the application hosting.

2 Implementation and components

Our application can be divided in frontend and backend:

- The frontend consists in *css*, *HTML* and *JavaScript* code hosted on AWS amplify handling the requests necessary to provide the service to the user.
- The backend is made up of 3 lambda functions handling the processing of the image and the management of storage resources.

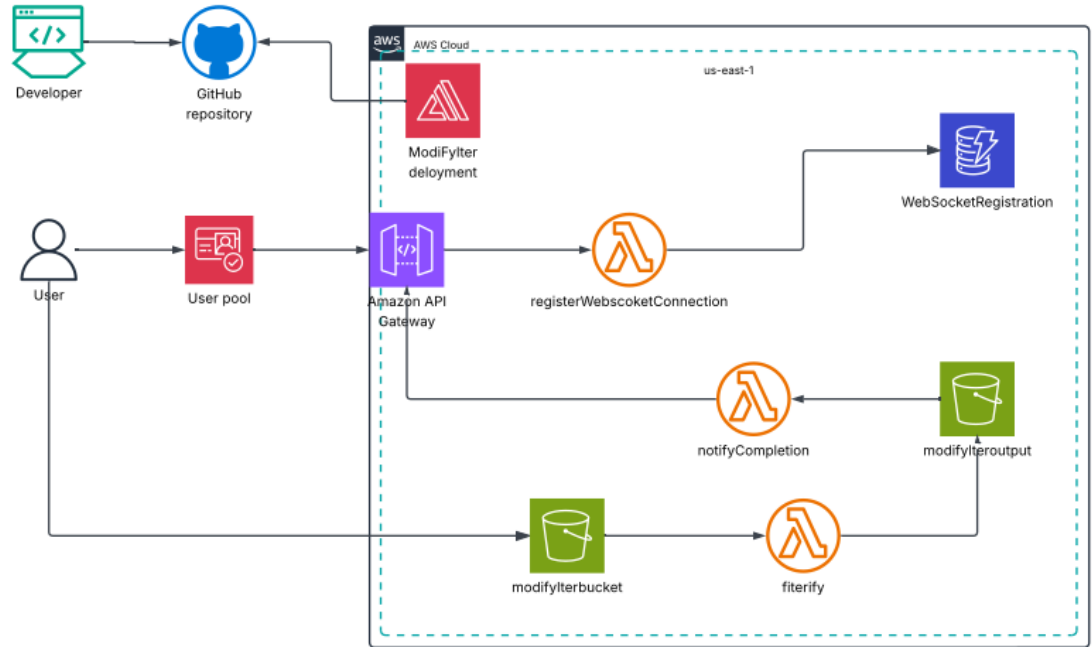


Figure 2: The architecture of the ModiFylter application.

We will first describe the flow of the application through the js code running in the frontend, hinting at which AWS services are being used and for which purpose. We will then delve further into how each component has been instantiated specifically for ModiFylter.

2.1 The frontend

The frontend follows a basic structure for building websites: a *css* file (`style.css`) for styling, an *HTML* file (`index.html`) for the positioning of elements in the page, and a *JavaScript* (`script.js`) file to handle the application logic and the requests to be sent to AWS. Since the *css* and *HTML* files are trivial and unrelated to the application, we will take about `script.js`. For interfacing with AWS we use the *AWS SDK for JavaScript v2.1659.0*

2.1.1 Page loading and authentication

As soon as the page is loaded, the code initializes some variables for later use, including the *identity pool ID* and the name of the region (*us-east-1*). We use `AWS.config.update()` to bind the session to the identity pool, and `AWS.config.credentials.get()` to retrieve the credentials that will be used to send the requests. The client also starts a web socket connection with AWS, which will be used to communicate meta information about the state of the process and identification information about the client (the *identity ID* obtained from the identity pool).

2.1.2 Image loading, filter selection and Modify

The user can interact with the application via a series of buttons and selectors:

- The **upload image** button opens up a file chooser interface, which allows to select the image that has to be filtered. The image is validated (only `.jpg` and `.png` extensions are accepted), a URL is created and the image is shown to the user.
- the **filter selection** button, once hovered on, shows a set of filters from which to choose from (for this version of the application, only "Black & white" and "Grayscale" are available).

Once both the image and the filters have been correctly selected, the image is processed (i.e. it is converted to base 64, and a JSON object for the request is prepared) and a new button, **modify**, appears. Once clicked, every other button is momentarily deactivated (to avoid inconsistencies) and the image is sent to AWS for elaboration. In particular it is loaded into an S3 bucket (`s3.upload`), which will trigger a lambda function for filtering the image and continuing the flow of the application.

We tried using `lambda.invoke()` in order to process the image directly without any intermediate step, but we noticed that it was impossible to send bigger images, since it had to be sent as HTTP payload. Loading the image to S3 and calling the lambda function AWS-side solves this problem.

2.1.3 Image download

The upload of the image on the S3 bucket triggers a lambda function that applies the requested filter to the image. The processed image is then stored in another bucket, and another lambda function retrieves the *connection ID* of the web socket opened beforehand, which has been paired with the *identity ID*. The *connection ID* is used to send a message to the client, notifying it that the image has been successfully processed and is ready for retrieval; the reception of this message triggers an `s3.getObject()` call to request the image and an `s3.deleteObject()` call to clean the S3 bucket.

At this point the result is shown on the page, and a download button is made visible to the user, so that the user can download the image. Subsequently,

the user is free to select another image and request another *modification*; the logic defined in `script.js` takes care of displaying the correct buttons at every action, avoiding inconsistencies or unforeseen circumstances.

2.2 AWS services

In the following, we describe the complete configuration of the backend of our application, i.e. which services we used and how we used them.

2.2.1 AWS Amplify

In order to deploy our application seamlessly without the need to manually configure everything, we decided to use AWS Amplify to host the frontend of our application. We uploaded the codebase of our frontend into a public repository (<https://github.com/twgever/ModiFylter/tree/main>) and told Amplify to deploy it. The website is hosted at <https://main.d36fucso8styeq.amplifyapp.com/>.

2.2.2 Amazon Cognito

Our frontend makes use of the *AWS SDK* to interact with the S3 buckets, hence it requires authentication in order to be authorized to do so. We created an identity pool (*IdentityUser*) to provide identities for each user accessing our web page. Each of these identities is associated with a unique *identity ID*, which we use internally to associate stored images with the corresponding user.

2.2.3 Amazon API Gateway

We created an API endpoint (*filtrifyConnection*) to handle communication via WebSocket. The user connects to its WebSocket URL and immediately calls the *register* method, which triggers the *registerWebsocketConnection* function, allowing to associate the user's WebSocket *connection ID* with its *identity ID*.

2.2.4 Amazon S3

We created two S3 (Simple Storage Service) buckets:

- **modifylterbucket:** To handle the images sent by the users. Uploading an object here triggers the *filterify* function. A JSON object is what's actually stored, so that the filter that has been chosen can be retrieved along with the image.
- **modifylteroutput:** To handle the processed images ready to be sent to the user. When a new image is uploaded here, the *notifyCompletion* function is triggered.

Both buckets have public access enabled and a CORS configuration allowing CRUD operations for the <https://main.d36fucso8styeq.amplifyapp.com/>

and `https://localhost:8000` origins. Lifecycle expiration rules are defined for both buckets in order to delete object after a certain amount of time, deleting objects that might remain inside the buckets due to internal errors.

2.2.5 AWS Lambda

We defined 3 lambda functions running python 3.13:

- **registerWebsocketConnection:** it is called when the user accesses the `/resource` URL on its WebSocket. It binds its *connection ID* with its *identity ID*, saving them in the same entry in a DynamoDB table.
- **filterify:** this is the function that processes the image, and it is triggered as soon as the image is uploaded on *modifyfilterbucket*. We defined a layer and added it to *filterify* in order to import *pillow*, the library we use for image processing. *filterify* retrieves the image from the S3 bucket via the *boto3* library, deletes it from the bucket and applies the specified filter, then it uploads the processed image on *modifyfilteroutput*, along with the user's *connection ID* (which it previously retrieved from the corresponding DynamoDB entry) for the following retrieval.
- **notifyCompletion:** this lambda function is triggered as soon as an object is uploaded on *modifyfilteroutput* (i.e. a processed image), and it simply sends a message to the user via WebSocket to notify that their image has been processed and is ready for retrieval.

Notably, we asked Amazon for a quota increase for the account's available concurrency. We went from a limit of 10 total reserved concurrency, to be able to reserve 200 concurrency just for *filterify*. After speaking with some colleagues that got their account banned during load testing, we considered this increase to be fundamental, as they reported that their account got banned because their actual concurrency exceeded the reserved concurrency.

2.2.6 Amazon DynamoDB

We created a DynamoDB table (*WebSocketRegistration*) to store user session information: the identity pool *identity ID* and WebSocket *connection ID*. This way, *notifyCompletion* can properly understand which *connection ID* is related to which user, and we avoided using the *connection ID* as identifier for the image in the S3 buckets or having to store the *connection ID* along with the image data. This means that further user session metadata for eventual updates of the application can be stored in this table, and there's no need to use other storing resources.

The partition key is the *identity ID*, but we created a global secondary index to enable retrieval via both IDs. We also defined a TTL for each entry, to delete unhandled entries that would otherwise remain indefinitely.

2.2.7 AWS IAM Identity Center

AWS Identity and Access Management Identity Center was used to provide every AWS resource the security roles they require for completion of their functions, for example: allowing S3 buckets to trigger lambda functions, allowing identities issued by the identity pool to upload and retrieve images to and from the S3 buckets, or even enabling lambda function logging onto Amazon CloudWatch.

2.2.8 Amazon CloudWatch

The usage of Amazon CloudWatch was essential for debugging the lambda functions. We leveraged the log groups to examine which errors were generated by which lambda function. Once the application was up and running, we logged the time it took to perform every major operations in every lambda function, which allowed us to identify some unnecessary payload overheads (resulting in time overhead) that we promptly corrected.

3 Evaluation

The objective of our evaluation process is described in the following question:

- How many request can our application comfortably serve without incurring in any major disruption?
- How many resources are needed to guarantee services for the heaviest workloads?
- Is there any bottleneck?
- What throughput can our application sustain?

We tried to answer this questions by imitating the behaviour of an hypothetical user via the node.js *puppeteer* library, which provides a framework for browser automation, and running the resulting script in *Docker* containers hosted on AWS, so that traffic is not flagged as suspicious and avoiding account banning (heavy workload coming from a single IP address outside of the AWS ecosystem can easily be suspected of being a DoS attack).

After presenting the data we used to conduct the evaluation, we describe how we implemented the test infrastructure on AWS, and then we show the results of our evaluation and perform an analysis.

3.1 Data

We collected two sets of 10 images each, the first set being *jpg* images, the second set being *png* images. We chose images in high definition with thousands of pixels in width and height, and then we scaled them down to lower resolutions. We ended up with a total of 80 images covering different resolutions and

extensions. In the following we show how we divided these images in categories and provide some data we can use for the evaluation:

category	average pixel number	average resolution	average image size	average result size
LD jpg	73 600	271x271	19,2 KB	26,6 KB
LD png	148 923	386x386	177,6 KB	38 ,1 KB
ND jpg	577 010	760x760	87,5 KB	190 KB
ND png	595 071	771x771	653,1 KB	143 ,1 KB
HD jpg	2,648 MP	1627x1627	325,9 KB	0,84 MB
HD png	2,373 MP	1541x1541	2,4 MB	? MB
HHD jpg	24,653 MP	4965x4965	2,7 MB	? MB
HHD png	22,321 MP	4275x4275	13,7 MB	? MB

Table 1: Image categories data. Note hat "MP" stands for megapixel

Please note the "average result size" column; this field calculates the mean size of the *ModiFylter* output for each category. As the filter is chosen randomly, we summed the size of the result for every image and for both, *grayscale* and *black & white* filters, and divided by 20.

3.2 Implementation

3.2.1 Puppeteer

The scripted behaviour of our *bot user* is defined in two js files importing the *puppeteer* library. We considered using other technologies, but the fact that the various requests to the AWS services are sent directly by the *script.js* of the web page means that we have not defined any API that can be independently tested, making browser automation mandatory in our case.

The first js file is *modifyfilterAction.js*, which defines a set of utility functions used to navigate *ModiFylter*:

- **openMF**: launches an headless browser and visits the URL of *ModiFylter*.
- **uploadImage**: clicks on the **upload image** button and uploads the image at the path in input.
- **selectRandomImage**: selects a random image from the specified folder and outputs its path.
- **selectRandomFilters**: selects a random filter out of the available ones under the **filter selection** button.
- **downloadImage**: waits for the image to be ready for download and clicks on the **download** button.
- **delay**: puts the program on hold for the specified amount of milliseconds.

- **randomDelay**: puts the program on hold for a random amount of milliseconds, from 1000 up to 4000.

These actions can be interleaved at will to build the most appropriate scenario, like a user that has a whole batch of images to process, or a user that changes idea continuously. For our case, a simple user that wants to process a single image is more than enough, and it is implemented in the *user.js* files, which defines a simple **user** function that puts everything together: it opens **ModiFylter**, selects a random image, uploads it, selects a random filter, clicks on **modify**, downloads the image when it is ready and deletes it (to avoid filling up the memory). Every action is interleaved by the **randomDelay** function, in order to emulate the time it takes a human user to click on the different buttons. The *user.js* file also provides an interface, through the **main()** function, that can be used to build the tests that we need by defining what happens in different phases. These are the possible arguments, in order:

- The resolution: LD,ND,HD,HHD
- The extension: *jpg* or *png*
- The download path
- How many users to create at each phase: [4,17,5] means 4 users in the 1st phase, 17 in the 2nd phase and 5 in the 3rd
- How long each phase has to last in milliseconds: [53 000, 1 000] means that the 1st phase should last 53 seconds and that the 2nd phase is 1 second long

For each phase, users are started in a uniform fashion, depending on the duration of the phase: if the 1st phase lasts 10 seconds and has to start 100 users, each user is going to be started 100 milliseconds after the previous one.

3.2.2 Docker

Since we wanted our *bot users* to run on AWS server, we need a way to specify our desired environment while also providing the necessary js code, the images for evaluation and the test defining arguments.

To do this we used *Docker*: we built an image (*twgever/user*) with 2 stages (to reduce its size). The first stage installs *puppeteer*, while the second stage installs chromium and starts a *Node* runtime that executes the *user.js* code. We uploaded the image on *Docker Hub*.

3.2.3 Amazon ECR

We created a private repository (*modifyfilter/test*) on Amazon Elastic Container Registry, where we pushed our Docker image, so that it is available to set up containers in AWS.

3.2.4 Amazon ECS

We used Amazon Elastic Container Service to handle the orchestration of the containers necessary to run the tests.

We created a cluster (*TestUser*) and configured it for AWS Fargate, so that we can run tasks without having to worry about obtaining compute capacity.

We also created a task definition (*SampleTest*), with *Launch type* set to AWS Fargate, *Operating system/Architecture* set to Linux/X86_64, set the *Image URI* to the URI of *modifyfilter/test* and specified to reserve 4 vCPUs and 16 GB of memory, so that each container can handle a moderate amount of chromium instances.

Once this is done, we're ready to carryout our tests, as we can easily override the `DOCKER_CMD` passed to each container when we run a task on the cluster, thus allowing us to run tests with as many users as we want.

3.3 Results

We performed a total of 6 different tests employing a conservative approach, starting from low resolution images with few requests and increasing both as we performed more tests, as to avoid account banning as much as we could, progressively testing the load size our application can serve. Sadly, we couldn't perform all of the tests we wanted: we incurred in a quota limit on the count of Fargate On-Demand vCPU, thus making it impossible to exceed a certain number of concurrent browser instances, even if headless. We were unable also to conduct tests on the HHD images, as the extra amount of time it takes to process them often triggers a puppeteer timeout that freezes the single users, compromising the test.

Nonetheless, we were able to prove that our application scales well, it can provide a modest level of service and doesn't require a lot of resources.

We defined all tests to have 2 minutes of *warm up*, 4 minutes of *ramp up*, 6 minutes of *steady* and 3 minutes of *ramp down*. Each test was performed 3 times, the results were collected via Amazon CloudWatch. We then extracted the mean and generated the plots with a custom Jupyter Notebook file (*metricsPlotting.ipynb*). Please note that the scale of the y-axis in the filterify metrics plotting is logarithmic.

For each test, we collected both, metrics regarding the duration of the lambda functions, to identify potential bottlenecks or inconsistencies, and metrics about the filterify function, as it is the function that has to handle the heaviest load.

3.3.1 Test 1 (LD jpg)

command override: "LD","jpg",".","[1,2,4,6,9,10,12,12,12,1,2,12,12,9,5,2]","[60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000]"

number of instances: 5

total requests: 600



We designed this first configuration to test the waters. Reaching a peak of 1 request per second we get the following:

maximum concurrency: 4

input throughput: 19,2 KB/s - 1,15 MB/m

output throughput: 26,6 KB/s - 1,57 MB/m

compute work: 0,074 MP/s - 4,42 MP/m

The plot shows that, as expected, `filterify` is the function that takes the most time to complete execution. All 3 functions take more time to complete at the beginning, probably due to AWS still having to initialize the necessary resources.

In the filterify plot, we notice how the number of concurrent executions increases with the number of invocation, which is completely normal behaviour.

3.3.2 Test 2 (LD png)

```
command override: "LD","png",".", "[2,3,5,7,11,15,18,18,18,  
18,18,18,13,8,3] ", "[60000,60000,60000,60000,60000,60000,6000  
0,60000,60000,60000,60000,60000,60000,60000,60000]"
```

number of instances: 5

peak requests per second: 1,5/s

total requests: 875



At 1,5 requests per second of low definition png images, we get:

maximum concurrency: 6

input throughput: 266,4 KB/s - 16 MB/m

output throughput: 57,15 KB/s - 3,43 MB/m

compute work: 0,223 MP/s - 13,4 MP/m

3.3.3 Test 3 (ND jpg)

command override: "ND", "jpg", ". ", "[2,4,7,11,16,20,24,24,24,24,24,24,15,7,3] ", "[60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000]"

number of instances: 5

peak requests per second: 2/s

total requests: 1 145



At 2 requests per second of normal definition jpg images, we still see no problem:

maximum concurrency: 5

input throughput: 175 KB/s - 10,5 MB/m

output throughput: 380 KB/s - 22,8 MB/m

compute work: 1,154 MP/s - 69,2 MP/m

3.3.4 Test 4 (ND png)

command override: "ND", "png", " . ", "[3,6,10,17,26,36,48,48,48,48,48,24,10,5] ", "[60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000]"

number of instances: 5

peak requests per second: 4/s

total requests: 2125



At 4 requests per seconds of normal definition png images, we exceed a concurrency of 10 for the first time, but this is not a problem, as our application can still handle the load:

maximum concurrency: 11

input throughput: 2,6 MB/s - 156,7 MB/m

output throughput: 0,57 MB/s - 34,3 MB/m

compute work: 2,38 MP/s - 142,81 MP/m

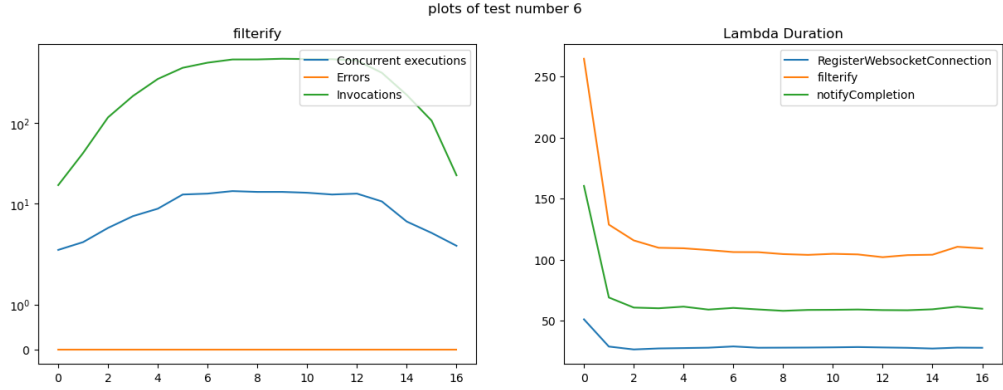
3.3.5 Test 5 (HD jpg)

command override: "HD", "jpg", ".", "[3,6,10,17,26,36,48,48,48,48,48,24,10,5]", "[60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000,60000]"

number of instances: 7

peak requests per second: 5,6/s

total requests: 2975



Interestingly, even if the throughput and compute work for the previous test with HD images are higher, this is the test that caused AWS to achieve the highest concurrency.

maximum concurrency: 16

input throughput: 201,6 KB/s - 12 MB/m

output throughput: 279,6 KB/s - 16,8 MB/m

compute work: 0,7 MP/s - 46,4 MP/m

4 Conclusion

In this report we presented *ModiFylter*, a novel application making use of the services provided on AWS to offer an unoriginal service in a scalable and maintainable way. Collecting metrics via a series of tests, we later showed that our application can handle different levels of throughput and compute work. While far from perfect, we believe our infrastructure constitutes a good starting point for an actual enterprise application.